

Introduction to Pattern Recognition

Spring 2026

10 ECTS Credits

Department of Information and Communication Technology

Faculty of Engineering and Science

About Me



- Assistant Professor
- Intelligent prognostics for remote industrial applications.
- Been in Norway since 2019
- Contact:

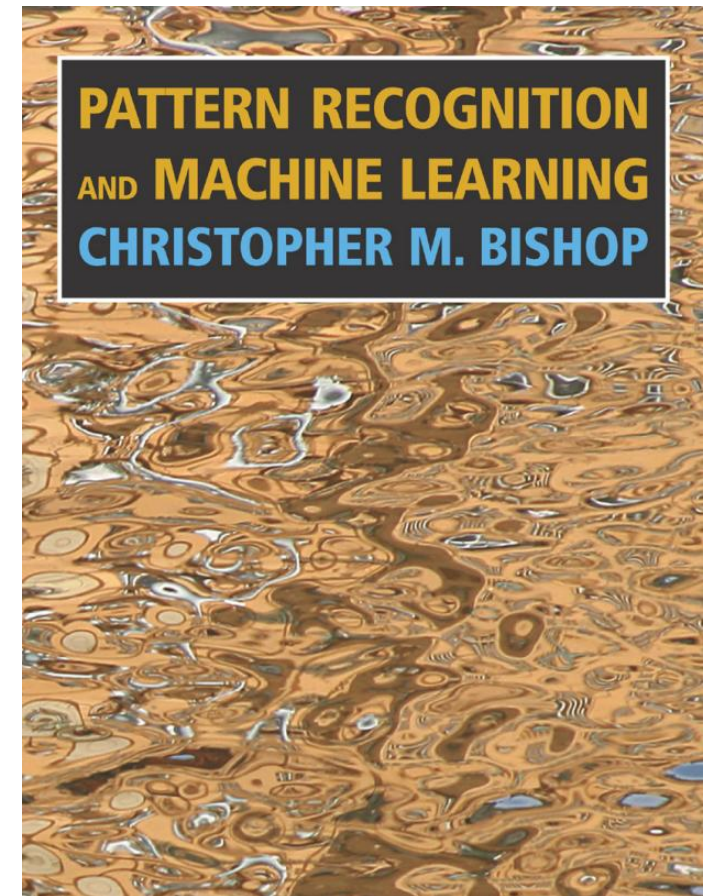
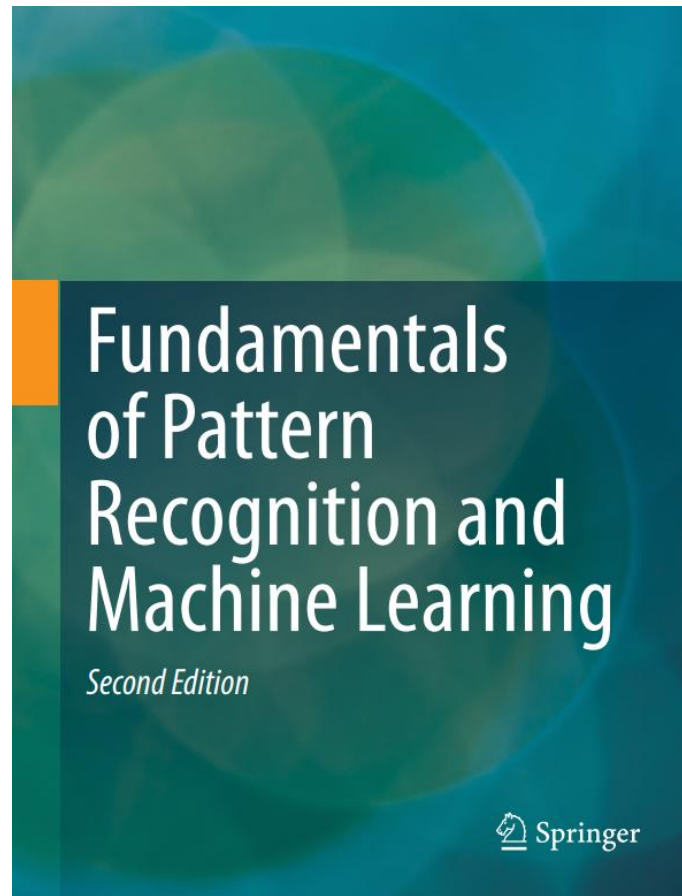
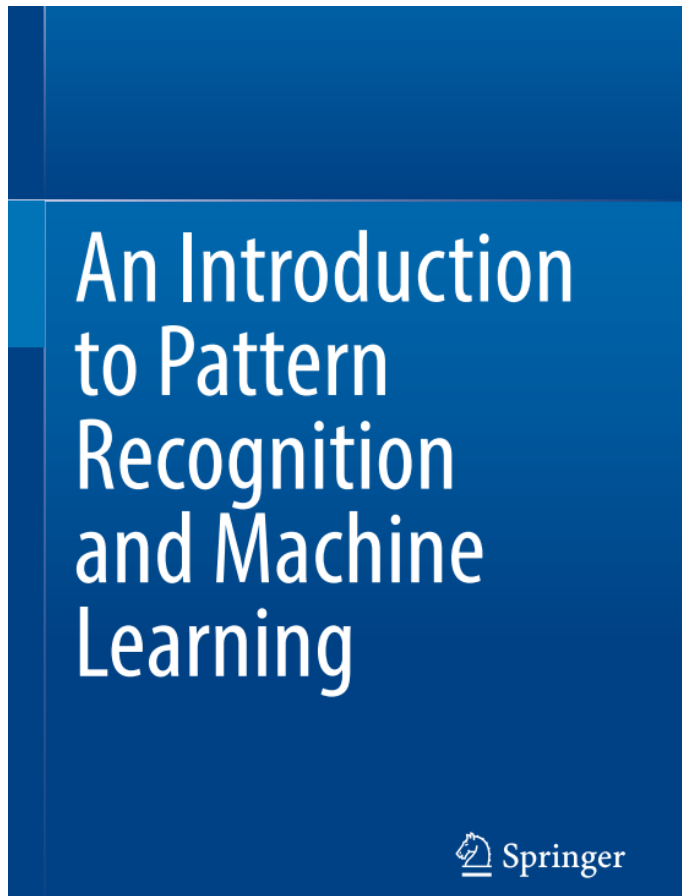
 manuel.s.mathew@uia.no

 A3 204

Course Objectives and Requirements

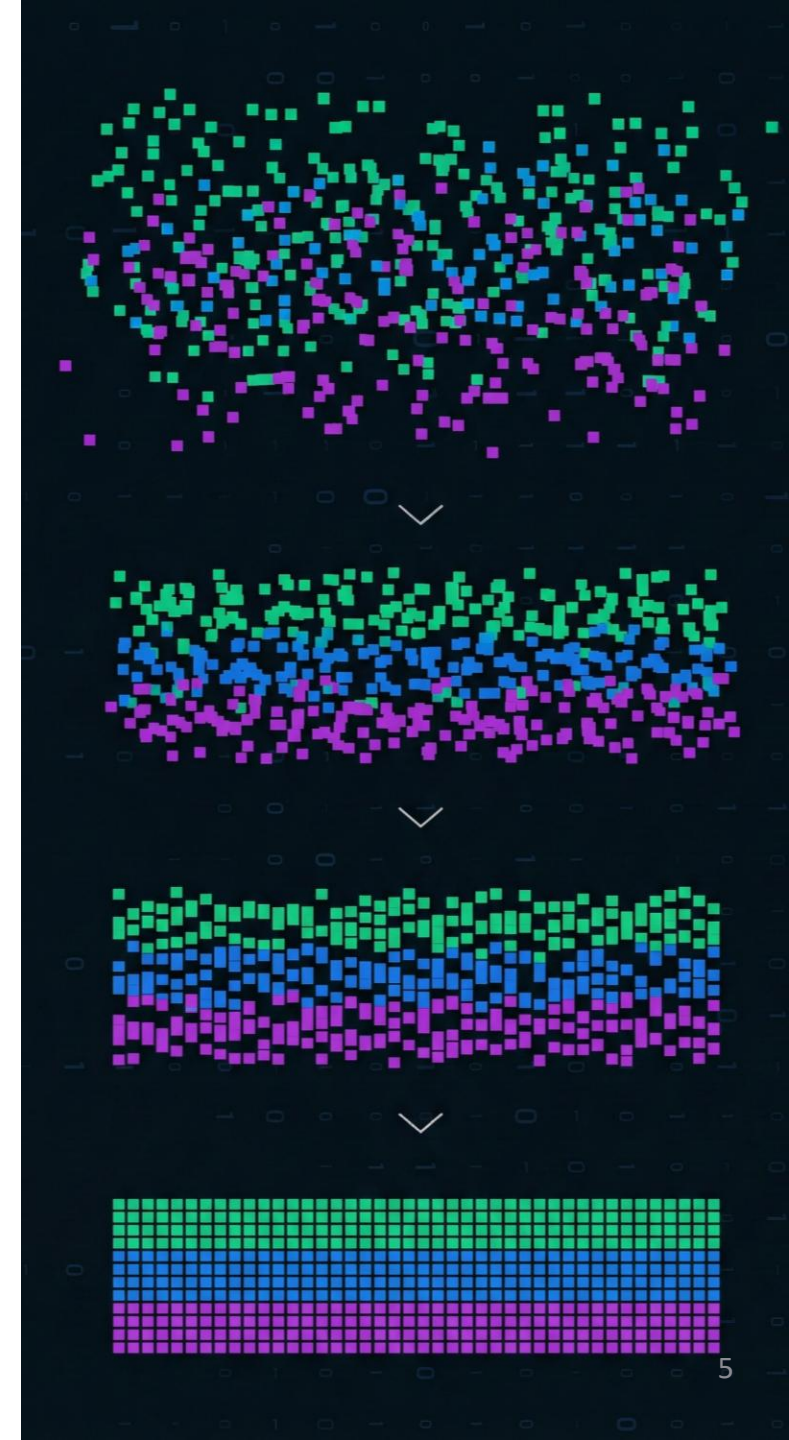
- Objectives:
 - Develop practical proficiency in pattern recognition.
 - Solve practical problems using pattern recognition.
 - Develop algorithms for this purpose using publicly available datasets.
 - Do basic data analysis and preprocessing of data.
- Requirements:
 - General knowledge of statistics.
 - Preliminary understanding of Python for data analytics.

Books that might be useful

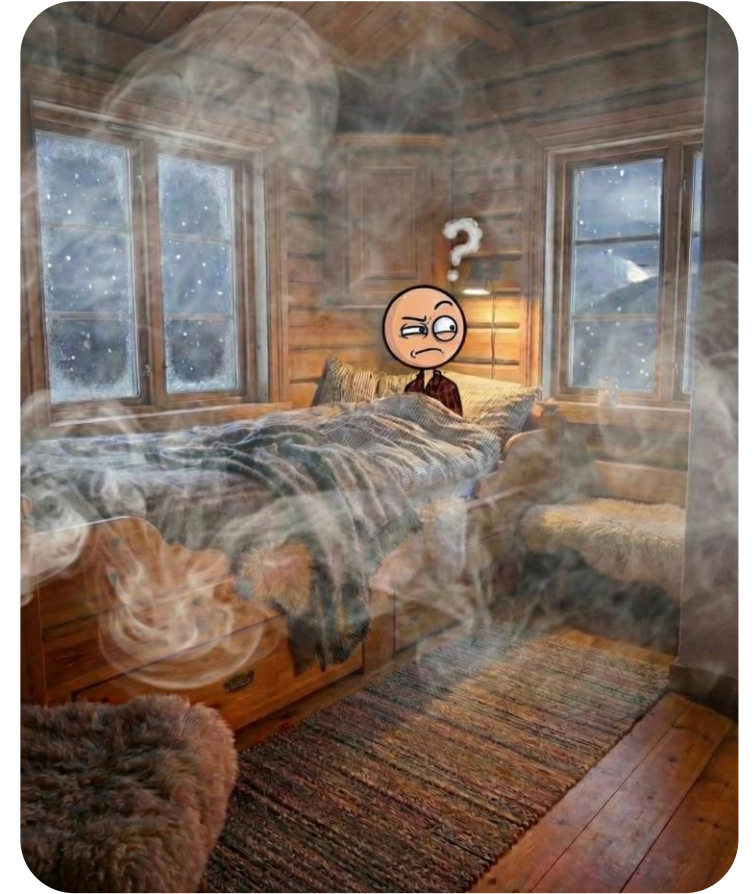


What are we doing here?

- What are patterns?
- Patterns are everywhere
- 600 million years ago, started with single celled organisms
- Environmental signals came into play for survival
- Predator and prey pattern recognition: a life/death scenario
- Social recognition through facial features
- Navigation using environmental cues.



A human experience



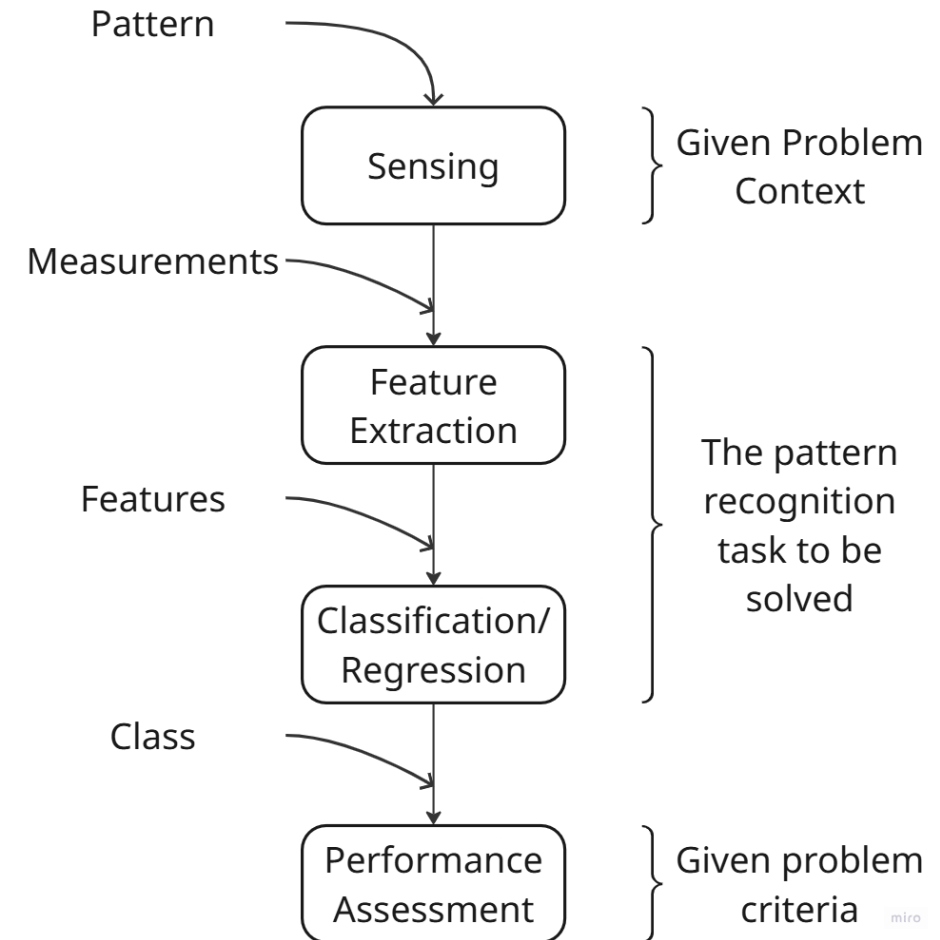
Automating Pattern Recognition

- Increase in processing power and large sets of data
- What are some applications of pattern recognition that you use daily?
- Some applications
 - Unlocking devices using biometrics
 - Recommender systems
 - Spam filtering in emails
 - Voice assistants
- Other applications
 - Diagnostics
 - Quality control systems



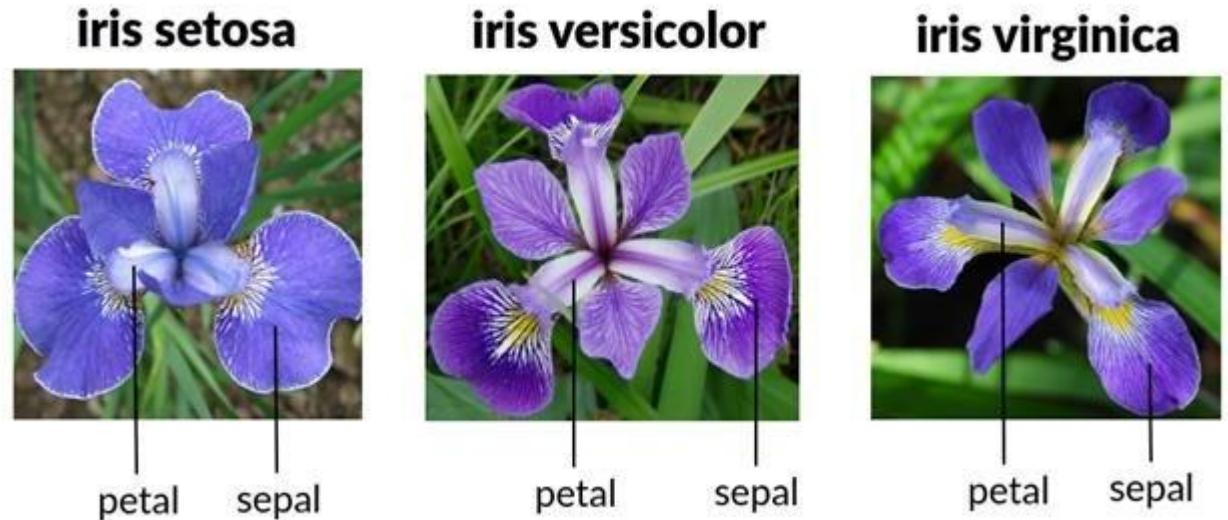
What is pattern recognition?

- “*Pattern recognition is a process by which some input is measured, analysed, and then classified as belonging to one of a set of classes*” – Paul Fieguth
- Two parts:
 - Systematic process of automatically identifying or analysing regularities or patterns within data.
 - Utilization of these patterns for categorical assignment (classification), continuous value predictions (regression).



An example

- Iris Dataset: a classic example.
- Three different plants to classify.
- Visible differences between these three flowers.
- How to automatically classify them?
- Sensing: gathering necessary data like sepal length, petal width, etc.
- We need some data preprocessing, but first we need to visualize our data.

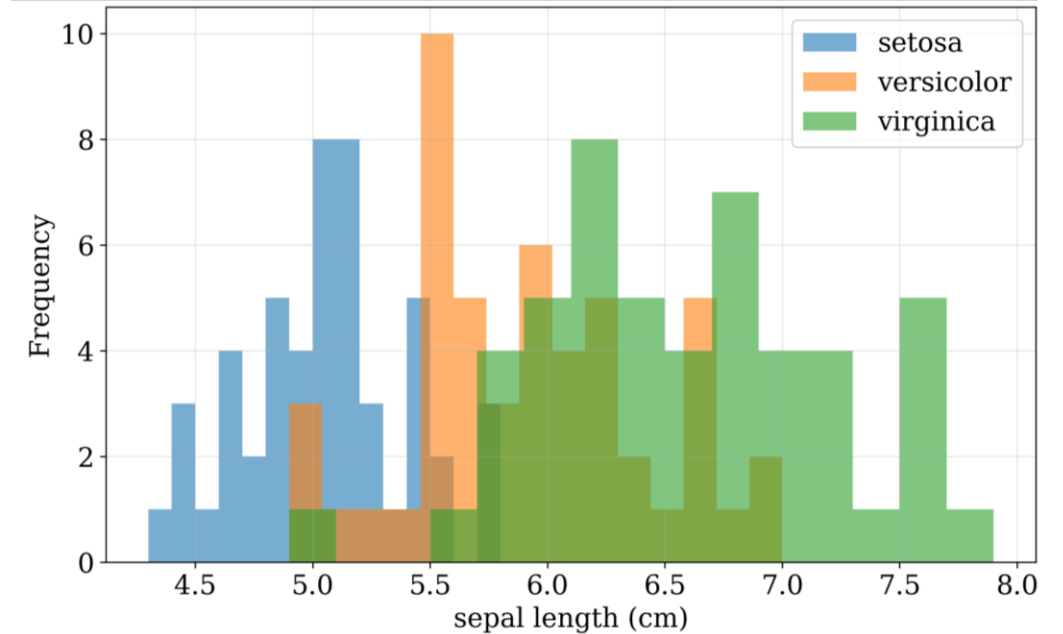
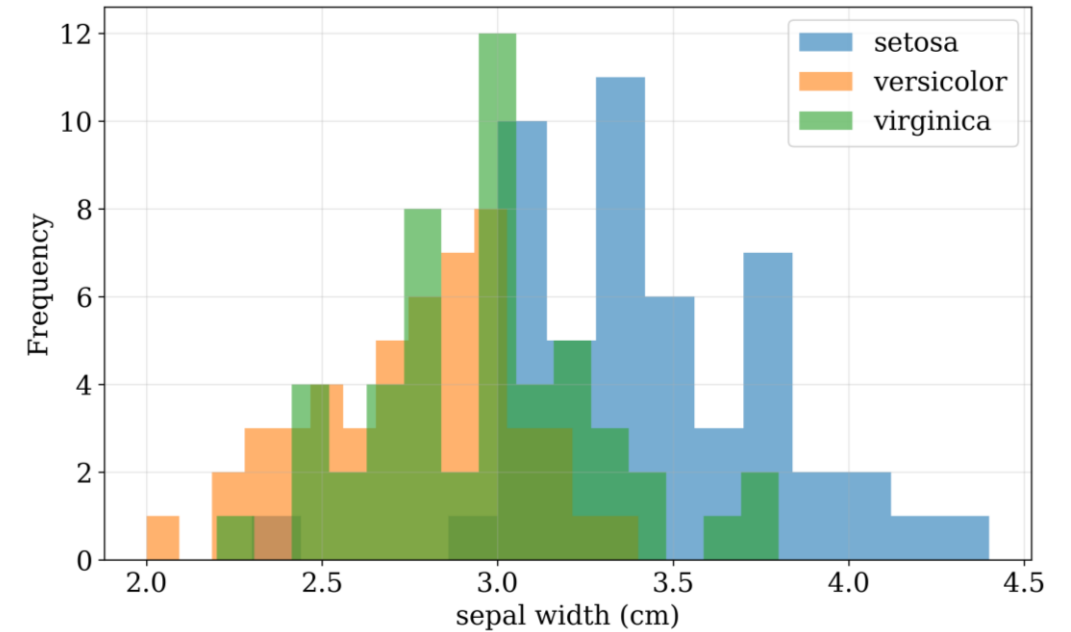


$C = \{\text{"Iris Setosa"}, \text{"Iris Versicolor"}, \text{"Iris Virginica"}\}$

$$x_i = \begin{bmatrix} \text{Sepal length} \\ \text{Sepal width} \\ \text{Petal length} \\ \text{Petal width} \end{bmatrix}$$

Single Feature Classification

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
6.1	2.8	4.7	1.2	versicolor
5.7	3.8	1.7	0.3	setosa
7.7	2.6	6.9	2.3	virginica
6.0	2.9	4.5	1.5	versicolor
6.8	2.8	4.8	1.4	versicolor
5.4	3.4	1.5	0.4	setosa
5.6	2.9	3.6	1.3	versicolor
6.9	3.1	5.1	2.3	virginica
6.2	2.2	4.5	1.5	versicolor
5.8	2.7	3.9	1.2	versicolor



Decision Threshold

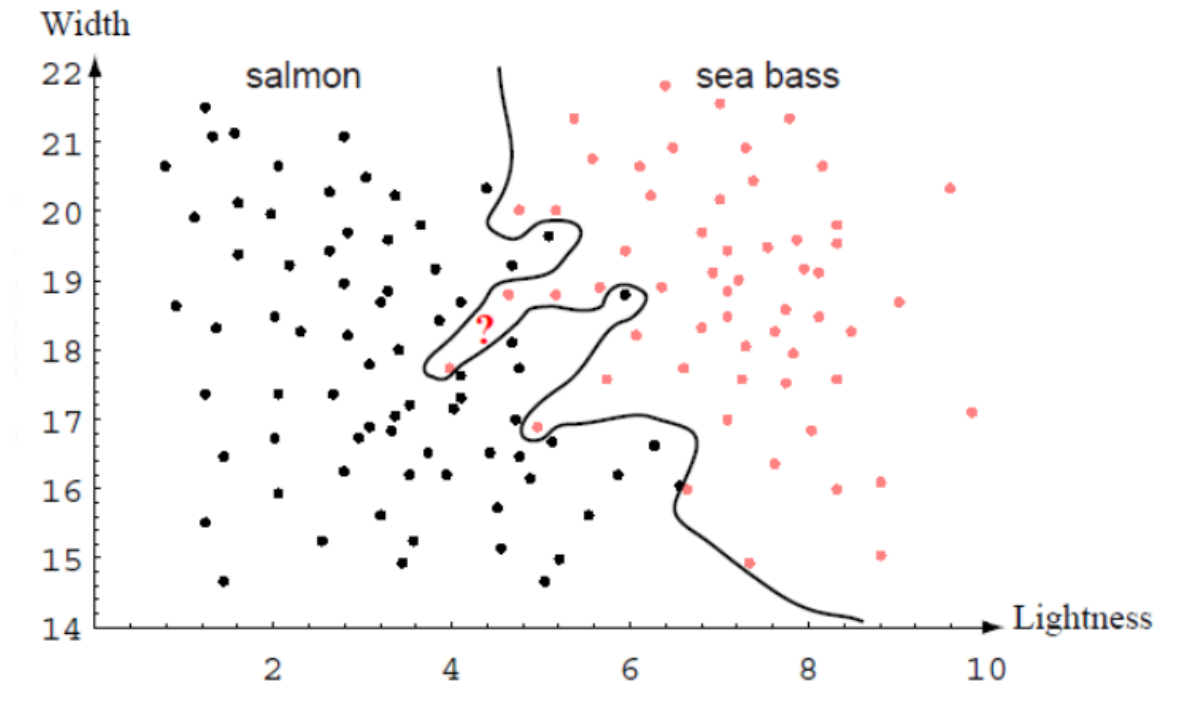
- If this single feature classification is acceptable, depends on the problem at hand.
- What is acceptable?
- Consider an automated fish classifier: salmon and seabass.
- With iris dataset, where the classification is multi-class, based on a confusion matrix.

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

		Predicted Class		
		setosa	versicolor	virginica
Actual Class	setosa	14	21	15
	versicolor	21	15	14
	virginica	17	15	18

Two feature classification

- Scatter plot based on two features.
- Not so easy to identify the **decision boundary** visually.
- The lines serves as the decision boundary of our classifier.
- The error in classification is much less with two feature classification.
- What if we give it a more complicated model. E.g. Salmon vs. Seabass.
- Perfect classification, but issues with **generalization** due to poor performance on future patterns.

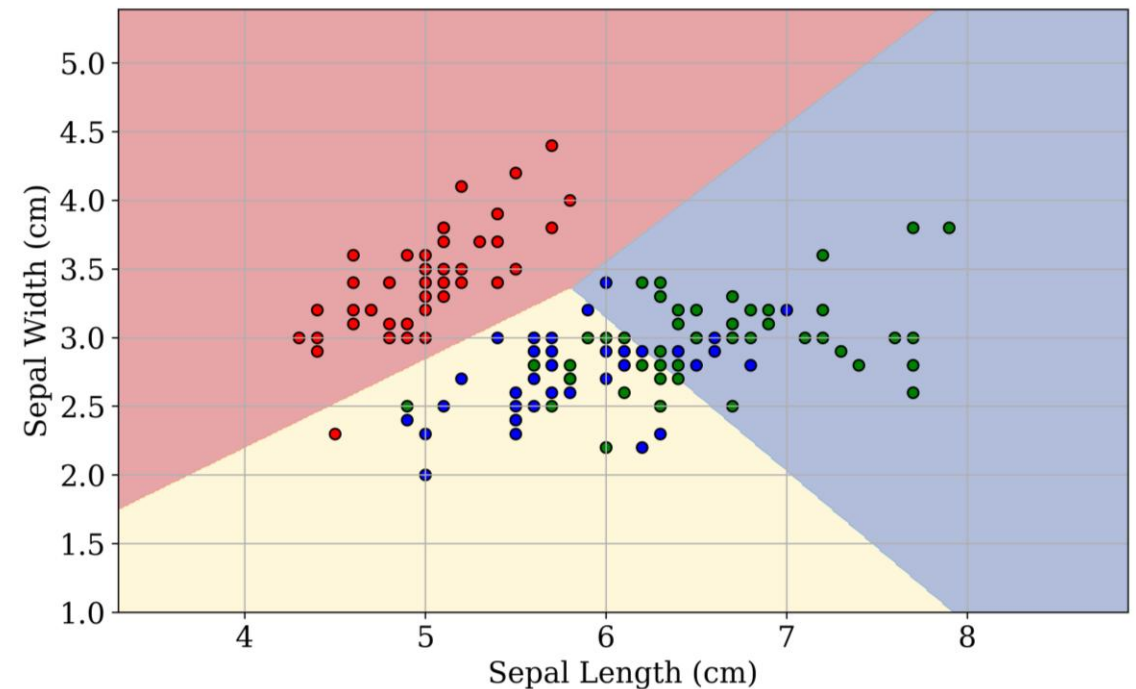


Questions ahead

- What if we add **more features**?
 - Is it **too expensive** to collect more data or computationally?
 - Does it provide **little to no improvement** to the model?
- Having few features
 - Simple, explainable, and faster
 - Lower memory consumption
- Does adding more data (**training samples**) improve the performance?
- What if we add more training samples and increase the **model complexity**?
- How do we know when we have arrived at the “**best classifier**”?

The optimal model

- This is why we don't use all our data to train a classifier.
- We divide that available dataset into **train** and **validation** datasets.
- The optimal model would perform better on both train and validation datasets.



First steps to an optimal model

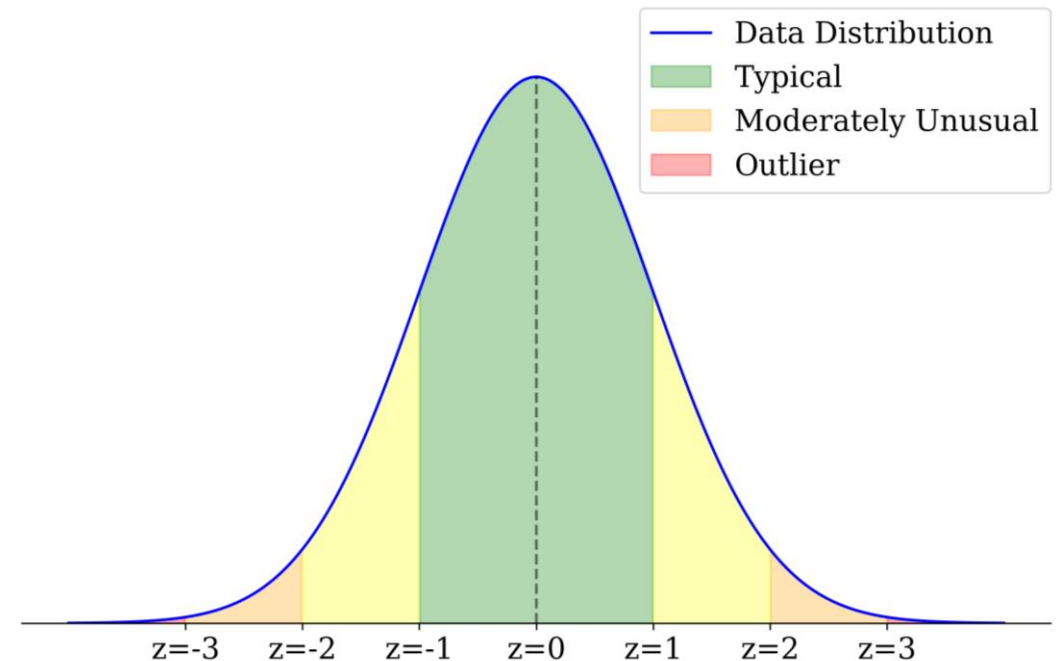
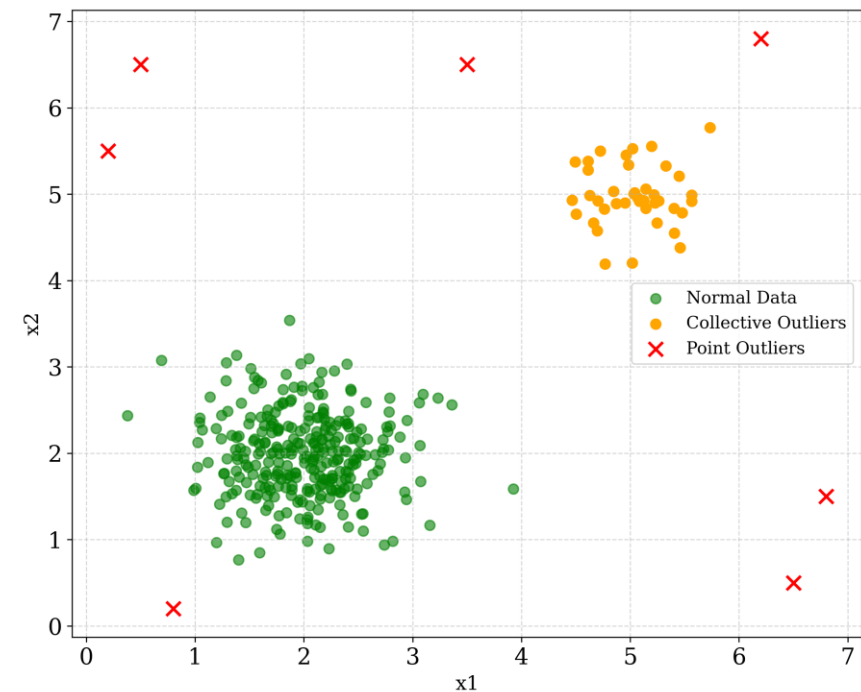
- Display some lines of data to get an idea.
 - What are the variables?
 - What are the types of these variables?
 - What is the size of your dataset?
- Are there any data points/variables that are:
 - missing?
 - shouldn't be there?
 - of no significance to the problem at hand?
- What are the main characteristics of the available data?
 - Statistical features like mean, median, etc.
- How does my data look like in a plot?
- What is my task?

Feature Extraction

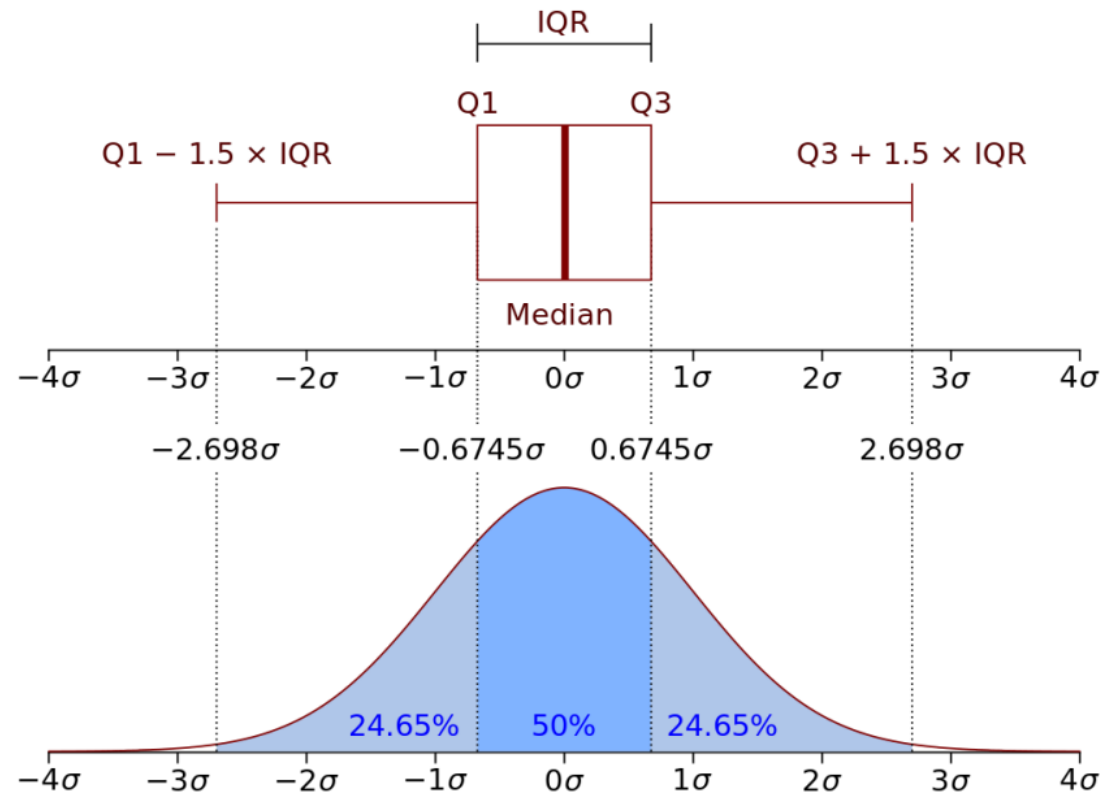
- Transforms raw dataset into meaningful representation by identifying and extracting the most relevant characteristics.
- Reducing **dimensionality** of the problem.
- Creating features in which patterns are clearly distinguished.
- It is more **problem and domain dependent**.
- How many features should we use?
- Is there a way to learn the best features automatically?
- Different methods:
 - Statistical methods – mean, median, standard deviation, correlation analysis, etc.
 - Dimensionality reduction – principal component analysis (PCA), linear discriminant analysis, etc.
 - Domain-specific techniques – edge detection in images, word embeddings for text

Outliers

- An **outlier** is a data point that deviates significantly relative to the rest of the dataset.
- Types:
 - Point outliers
 - Collective outliers
 - Contextual outliers
- Detection: Z-score analysis, interquartile range (IQR), DBSCAN, etc.
- Treatment:
 - Thoroughly analyse the cause
 - Consider domain knowledge when deciding treatment



Outlier detection using IQR



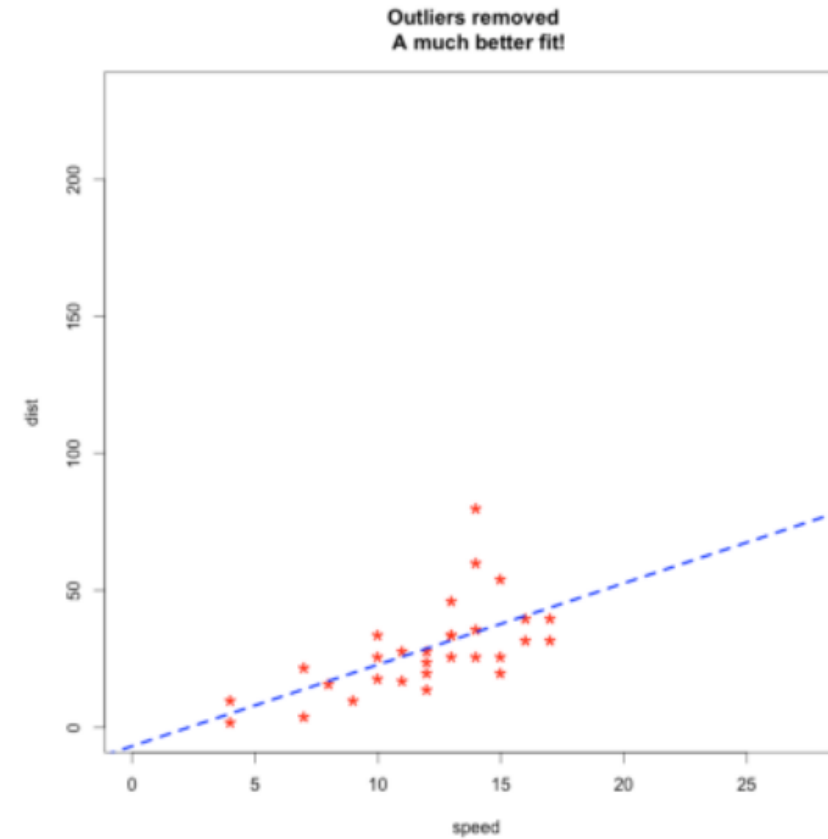
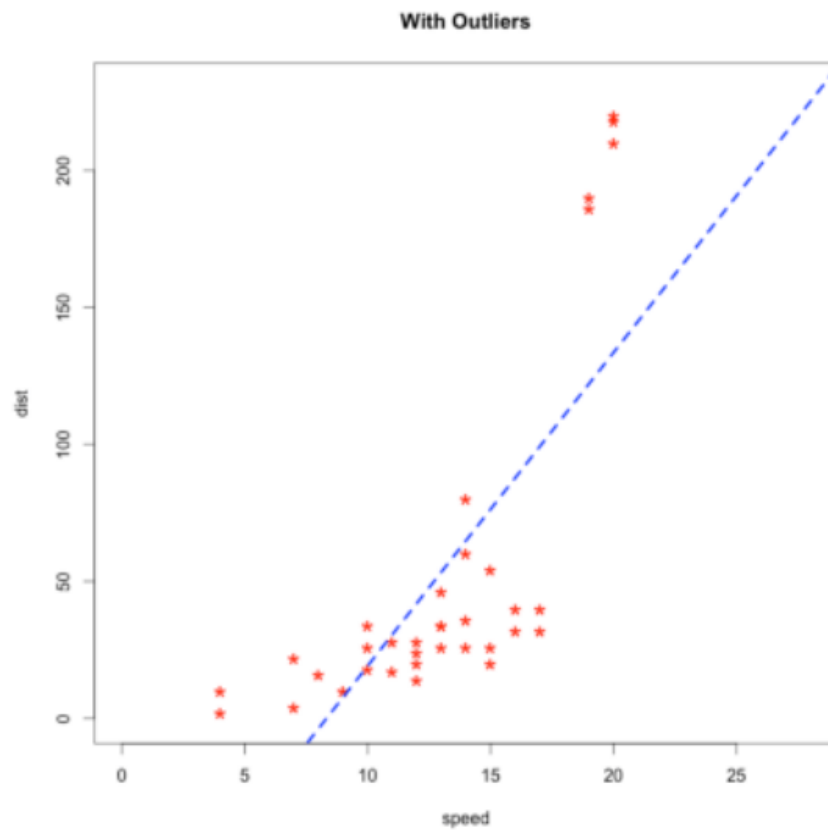
OUTLIER

$$x < Q1 - 1.5 \cdot IQR$$

OR

$$x > Q3 + 1.5 \cdot IQR$$

Impact of outliers



Missing Data

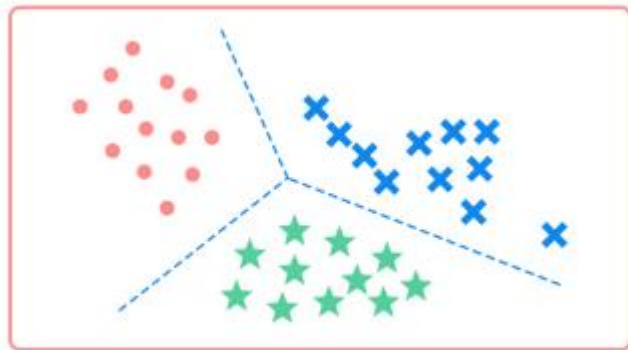
- Due to observation errors, instrumentation errors, etc.
- Types:
 - Missing completely at random (MCAR)
 - Missing at random (MAR): Explained by observed data.
 - Missing not at random (MNAR): Due to Unobserved Values.
 - Structurally Missing: There is no value.
- Treatment:
 - Deletion methods
 - Imputation methods: statistical, K-nearest neighbours, etc.

x_1	x_2	C
52	0.182	D
NaN	NaN	B
15	0.304	B
72	0.525	NaN
61	0.432	A
21	NaN	B
83	0.612	NaN
NaN	0.139	B
75	0.292	D
75	0.366	D

Types of Learning

Supervised Learning

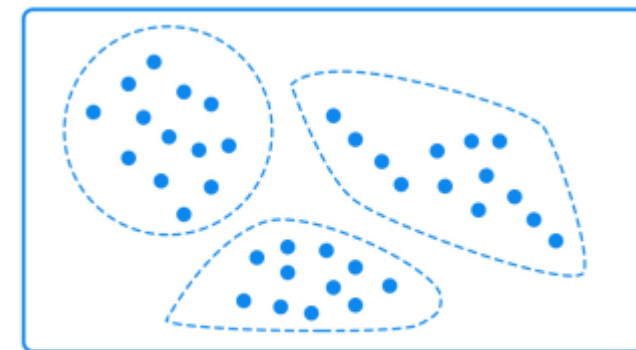
- Labelled input data.
- A feedback mechanism
- Predicting future outcomes
- Could be expensive



Supervised learning

Unsupervised Learning

- Input data is unlabelled
- No feedback mechanism
- Predicting relationship between the elements of the dataset



Unsupervised learning

Mathematical Setting

- In supervised learning, the problem is observed (summarized) into a vector of measurements known as a **feature vector** and a target:

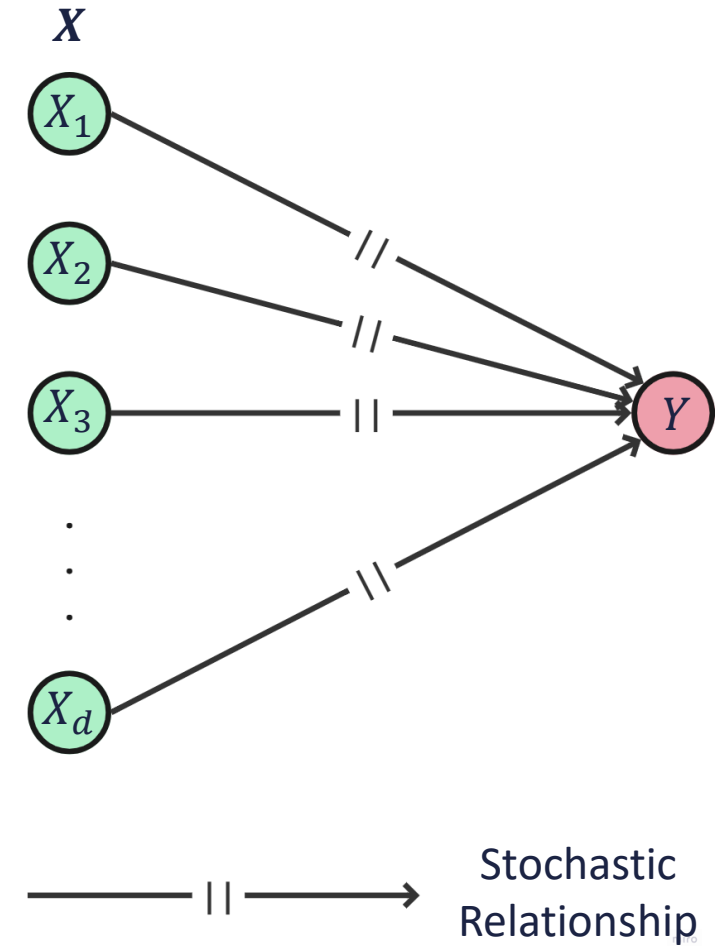
$$\mathbf{X} \in \mathbb{R}^d, \quad Y \in \mathbb{R}$$

- The relationship between these two are rarely deterministic. i.e. often there is no function such that:

$$Y = f(\mathbf{X})$$

- The relationship between $\mathbf{X}$ and Y can be specified by a joint feature-target distribution:

$$P_{\mathbf{X},Y}: \mathbf{X} \rightarrow Y$$



Prediction

- The objective: Predict Y given X .

- A **prediction rule** produces a **predictor**:

$$\psi: \mathbb{R}^d \rightarrow \mathbb{R}, \text{ such that } \psi(\mathbf{X}) \rightarrow Y.$$

- The predictor is designed using information about $P_{\mathbf{X},Y}$:
 - Direct knowledge,
 - Indirect knowledge: through an independent, and identically distributed (i.i.d.) sample (training data):

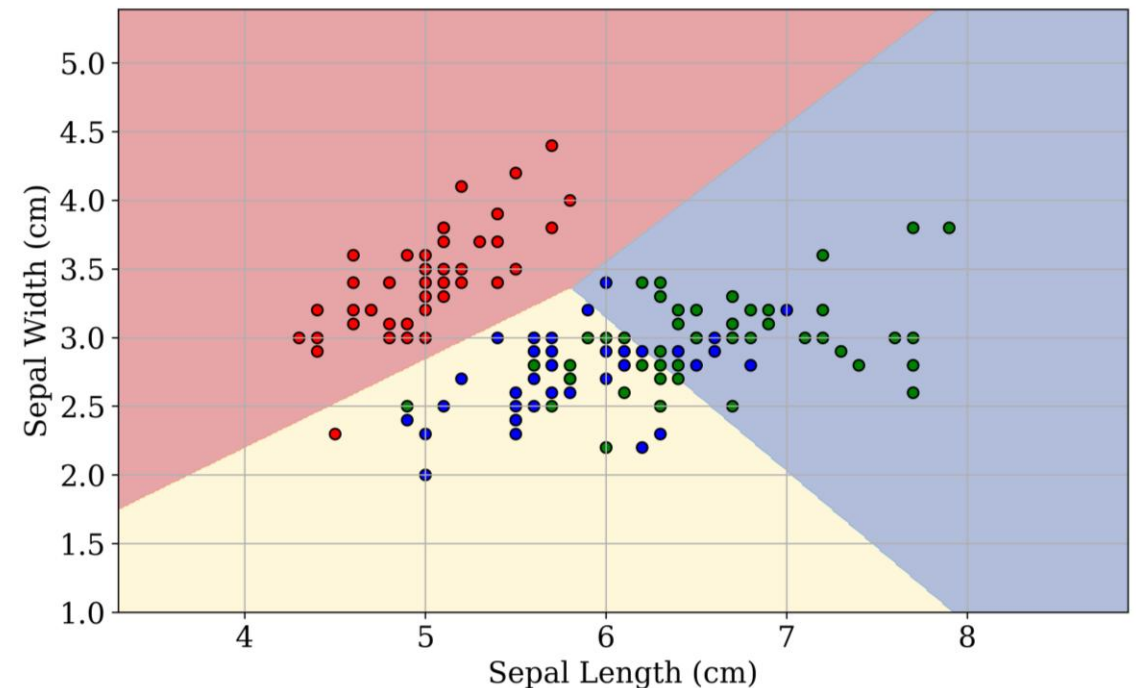
$$S_n = \{(\mathbf{X}_1, Y_1), (\mathbf{X}_2, Y_2), \dots, (\mathbf{X}_n, Y_n)\}$$

- In theory, with full knowledge of $P_{\mathbf{X},Y}$, an optimal predictor $\psi^*(\mathbf{X})$ can be designed.
- In practice, we use the sample S_n to design the predictor ψ_n .

$$n \rightarrow \infty, \quad \psi_n \rightarrow \psi^*$$

Costs and Risks

- There is always a cost and risk associated with each decision we make with the help of a predictor.
- These are higher towards the decision boundary.
- How can these costs and risks associated with prediction be integrated into the model?
- How about a penalty for making the wrong prediction?



Prediction Error

- The penalty signal to improve prediction, specified as a loss function:

$$\ell: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$$

- Measures the distance between predicted value $\psi(\mathbf{X})$ and the target Y . E.g.:

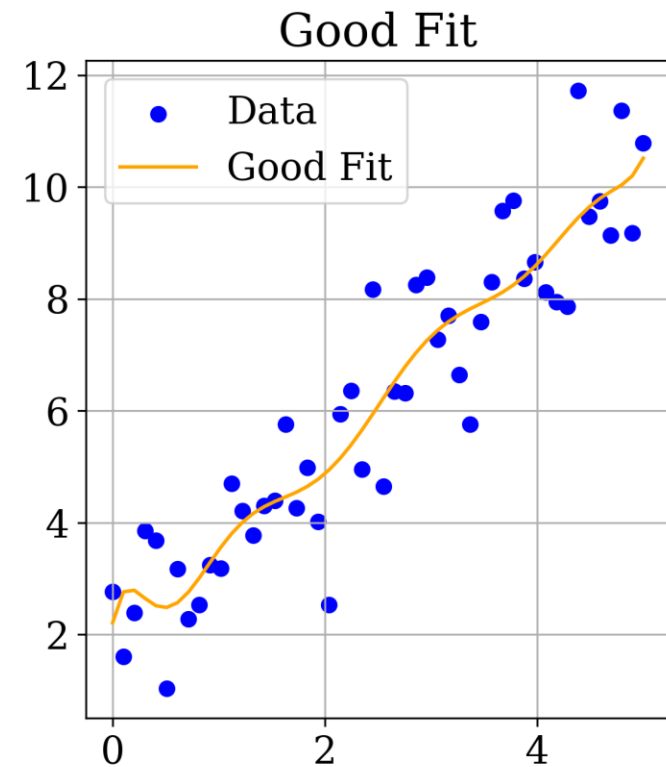
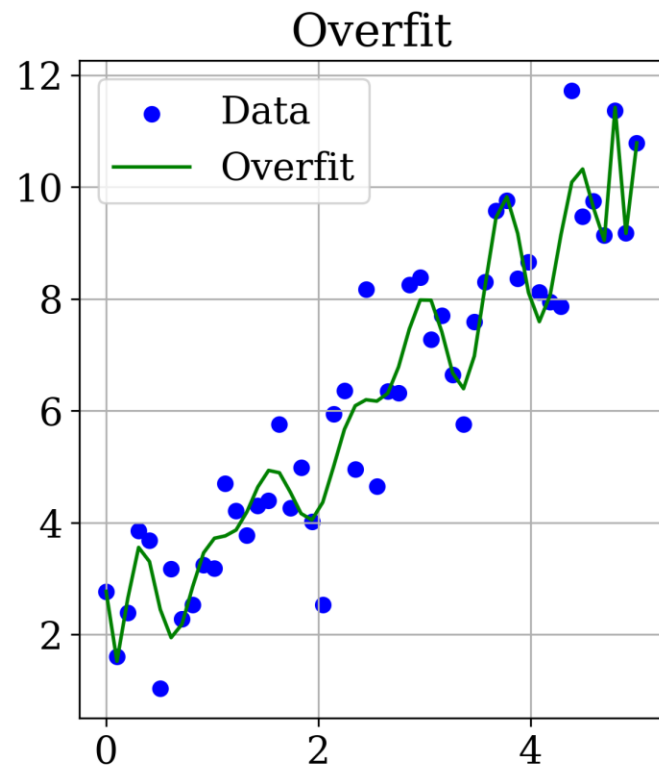
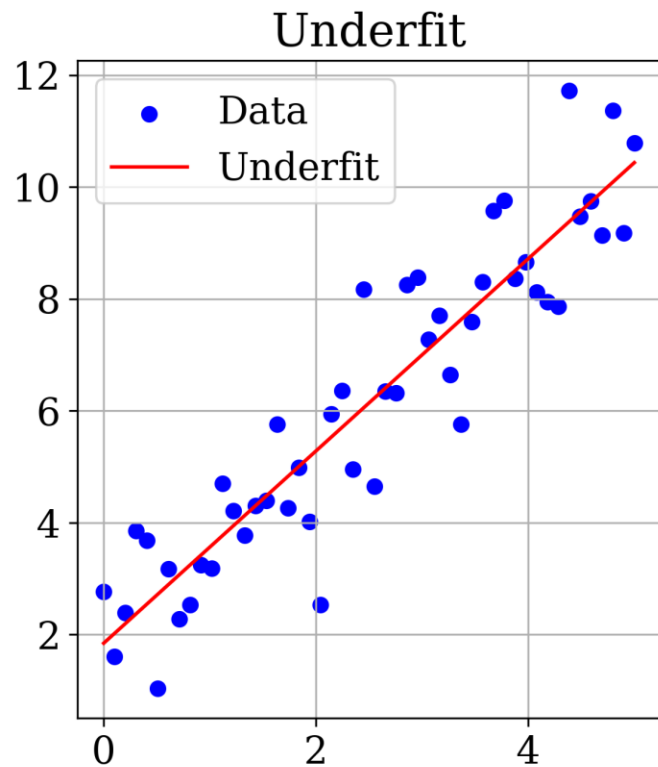
- Quadratic loss: $\ell(\psi(\mathbf{X}), Y) = (Y - \psi(\mathbf{X}))^2$,
- Absolute difference loss: $\ell(\psi(\mathbf{X}), Y) = |Y - \psi(\mathbf{X})|$,
- Misclassification loss:

$$\ell(\psi(\mathbf{X}), Y) = I_{Y \neq \psi(\mathbf{X})} = \begin{cases} 1, & Y \neq \psi(\mathbf{X}), \\ 0, & Y = \psi(\mathbf{X}). \end{cases}$$

- Since we are dealing with stochastic relationships, the loss of predictor (**prediction error**):

$$L[\psi] = E[\ell(\psi(\mathbf{X}), Y)].$$

Underfitting vs Overfitting

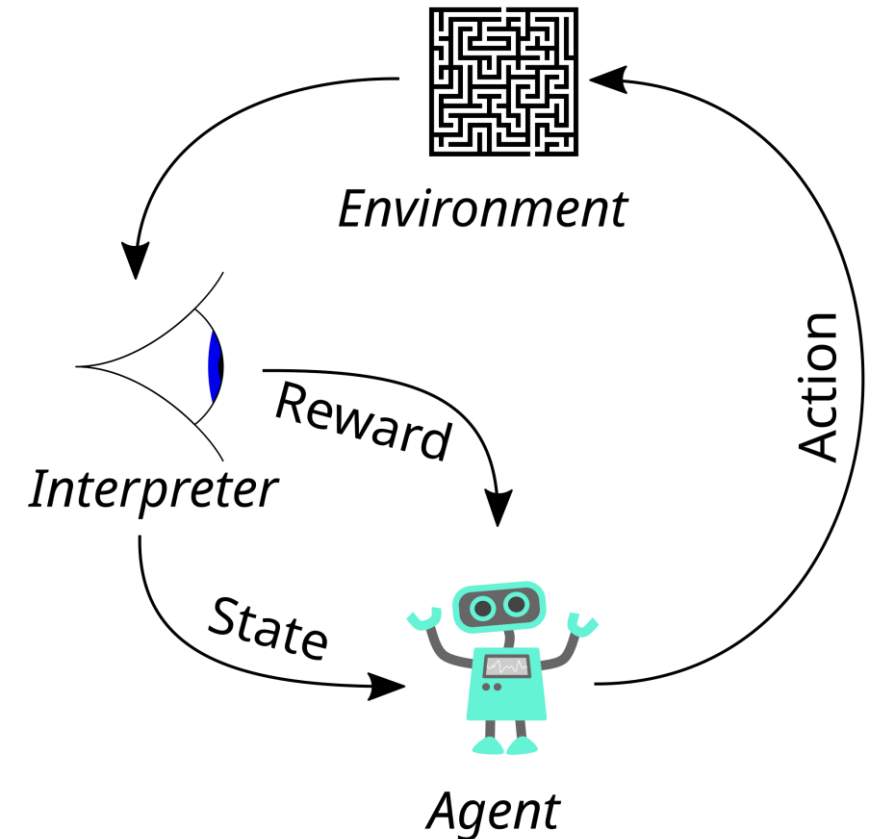


Unsupervised Learning

- Finding patterns and structure without labels/target variables.
- **Feature Vector:** $X \in \mathbb{R}^d$.
- **Dataset:** $\mathcal{D} = S_n = X$.
- **Objective** is to discover hidden structure:
 - Clustering,
 - Dimensionality reduction,
 - Density estimation.
- **Examples:** K-means clustering, principal component analysis (PCA), etc.
- **Performance assessment:** task-dependent metrics.

Reinforcement Learning

- Interpreter observes the **state** from an **environment**.
- Based on the **observation**, **agent** takes an **action** in the environment.
- Interpreter gives a **reward/penalty** to the agent.
- Agent updates **internal states** based on the reward.
- Iterates until an **optimal policy** is found.



YouTube – AI Learns to Walk